

SECTION 5

16-BIT BUS OPERATION

The following paragraphs describe control signal and bus operation for 16-bit bus operations during data transfer operations, bus arbitration, bus error and halt conditions, and reset operation. The 16-bit bus operation devices are the MC68000, MC68HC000, MC68010, and the MC68HC001 and MC68EC000 in 16-bit mode. The MC68HC001 and MC68EC000 select 16-bit mode by pulling mode high or leave it floating during reset.

5.1 DATA TRANSFER OPERATIONS

Transfer of data between devices involves the following signals:

1. Address bus A1 through highest numbered address line
2. Data bus D0 through D15
3. Control signals

The address and data buses are separate parallel buses used to transfer data using an asynchronous bus structure. In all cases, the bus master must deskew all signals it issues at both the start and end of a bus cycle. In addition, the bus master must deskew the acknowledge and data signals from the slave device.

The following paragraphs describe the read, write, read-modify-write, and CPU space cycles. The indivisible read-modify-write cycle implements interlocked multiprocessor communications. A CPU space cycle is a special processor cycle.

5.1.1 Read Cycle

During a read cycle, the processor receives either one or two bytes of data from the memory or from a peripheral device. If the instruction specifies a word or long-word operation, the MC68000, MC68HC000, MC68HC001, MC68EC000, or MC68010 processor reads both upper and lower bytes simultaneously by asserting both upper and lower data strobes. When the instruction specifies byte operation, the processor uses the internal A0 bit to determine which byte to read and issues the appropriate data strobe. When A0 equals zero, the upper data strobe is issued; when A0 equals one, the lower data strobe is issued. When the data is received, the processor internally positions the byte appropriately.

The word read-cycle flowchart is shown in Figure 5-1 and the byte read-cycle flowchart is shown in Figure 5-2. The read and write cycle timing is shown in Figure 5-3 and the word and byte read-cycle timing diagram is shown in Figure 5-4.

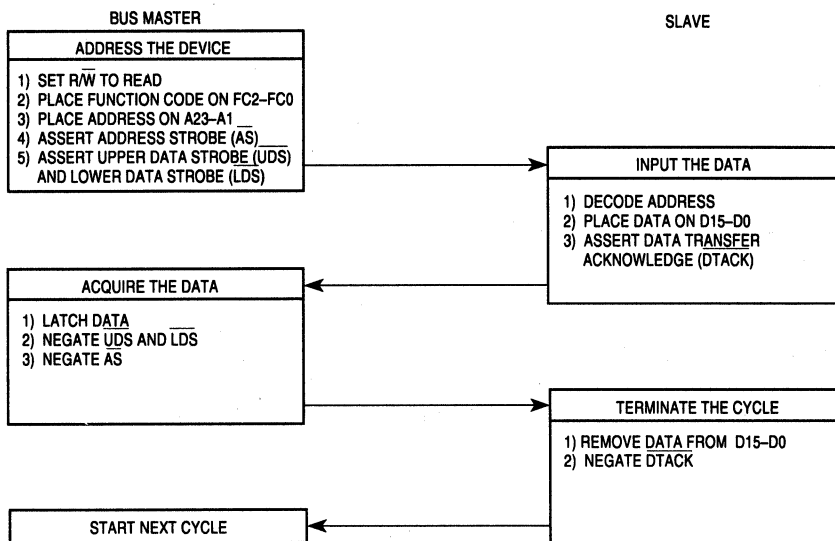


Figure 5-1. Word Read-Cycle Flowchart

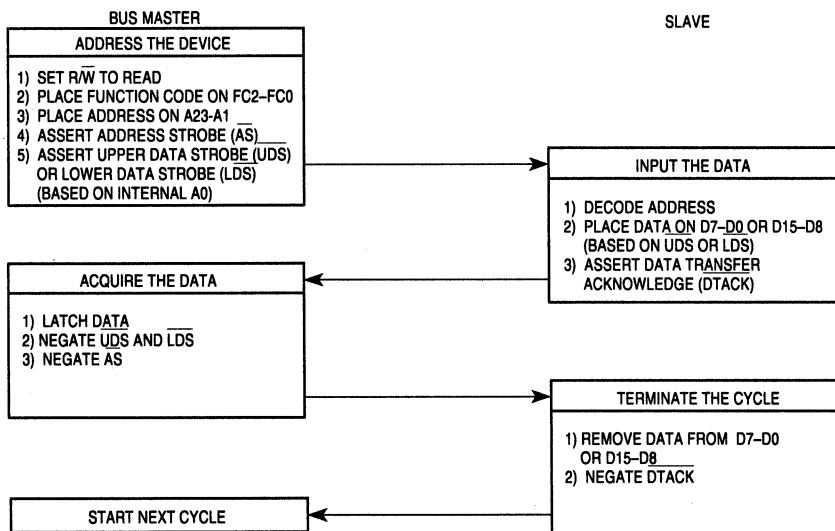


Figure 5-2. Byte Read-Cycle Flowchart

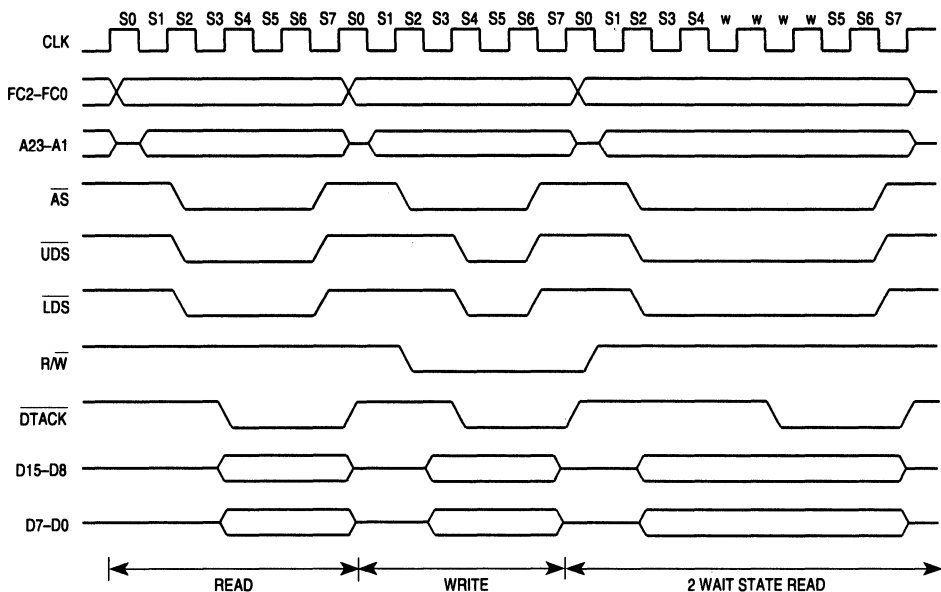


Figure 5-3. Read and Write-Cycle Timing Diagram

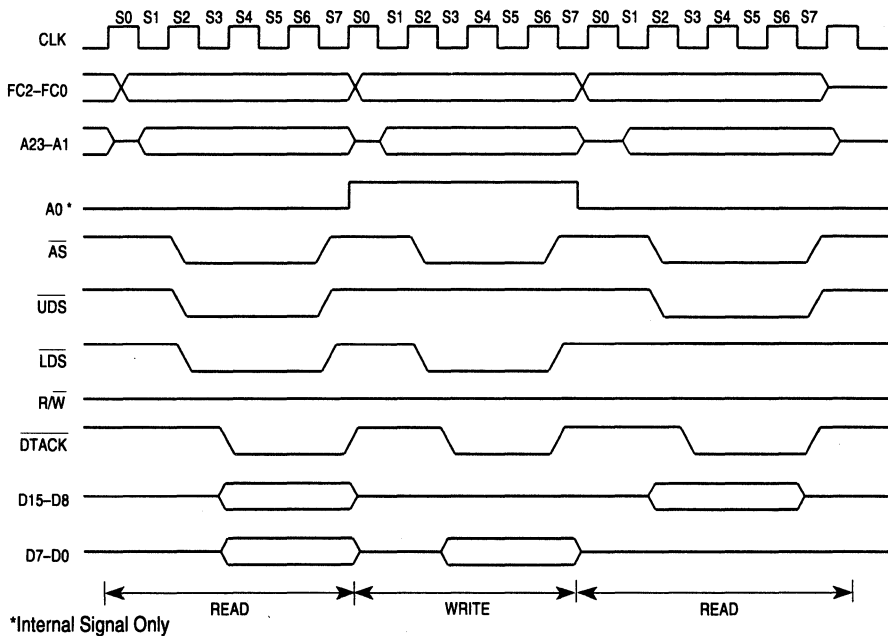


Figure 5-4. Word and Byte Read-Cycle Timing Diagram

A bus cycle consists of eight states. The various signals are asserted during specific states of a read cycle, as follows:

- STATE 0** The read cycle starts in state 0 (S0). The processor places valid function codes on FC0–FC2 and drives R/W high to identify a read cycle.
- STATE 1** Entering state 1 (S1), the processor drives a valid address on the address bus.
- STATE 2** On the rising edge of state 2 (S2), the processor asserts $\overline{AS}$ and $\overline{UDS}$, $\overline{LDS}$, or $\overline{DS}$.
- STATE 3** During state 3 (S3), no bus signals are altered.
- STATE 4** During state 4 (S4), the processor waits for a cycle termination signal ($\overline{DTACK}$ or $\overline{BERR}$) or $\overline{VPA}$, an M6800 peripheral signal. When $\overline{VPA}$ is asserted during S4, the cycle becomes a peripheral cycle (refer to **Appendix B M6800 Peripheral Interface**). If neither termination signal is asserted before the falling edge at the end of S4, the processor inserts wait states (full clock cycles) until either $\overline{DTACK}$ or $\overline{BERR}$ is asserted.
- STATE 5** During state 5 (S5), no bus signals are altered.
- STATE 6** During state 6 (S6), data from the device is driven onto the data bus.
- STATE 7** On the falling edge of the clock entering state 7 (S7), the processor latches data from the addressed device and negates $\overline{AS}$, $\overline{UDS}$, and $\overline{LDS}$. At the rising edge of S7, the processor places the address bus in the high-impedance state. The device negates $\overline{DTACK}$ or $\overline{BERR}$ at this time.

NOTE

During an active bus cycle, $\overline{VPA}$ and $\overline{BERR}$ are sampled on every falling edge of the clock beginning with S4, and data is latched on the falling edge of S6 during a read cycle. The bus cycle terminates in S7, except when $\overline{BERR}$ is asserted in the absence of $\overline{DTACK}$. In that case, the bus cycle terminates one clock cycle later in S9.

5.1.2 Write Cycle

During a write cycle, the processor sends bytes of data to the memory or peripheral device. If the instruction specifies a word operation, the processor issues both $\overline{UDS}$ and $\overline{LDS}$ and writes both bytes. When the instruction specifies a byte operation, the processor uses the internal A0 bit to determine which byte to write and issues the appropriate data strobe. When the A0 bit equals zero, $\overline{UDS}$ is asserted; when the A0 bit equals one, $\overline{LDS}$ is asserted.

The word and byte write-cycle timing diagram and flowcharts in Figures 5-5, 5-6, and 5-7 applies directly to the MC68000, the MC68HC000, the MC68HC001 (in 16-bit mode), the MC68EC000 (in 16-bit mode), and the MC68010.

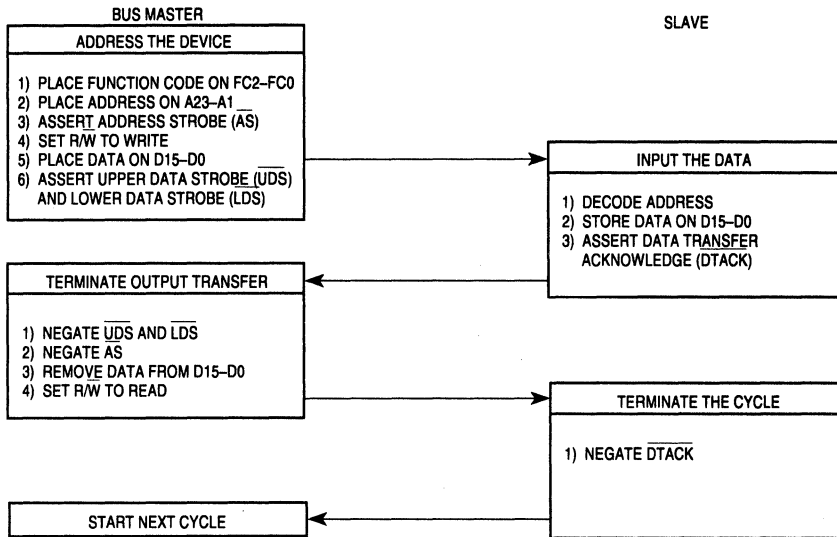


Figure 5-5. Word Write-Cycle Flowchart

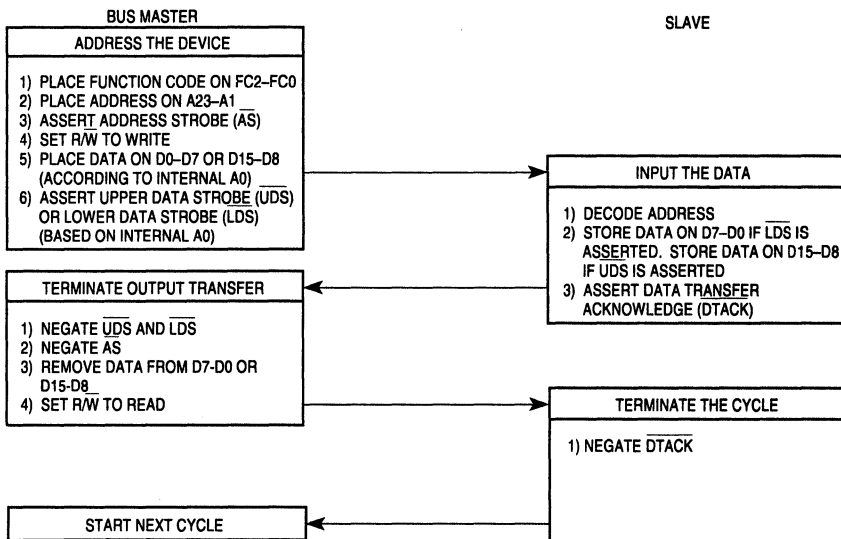


Figure 5-6. Byte Write-Cycle Flowchart

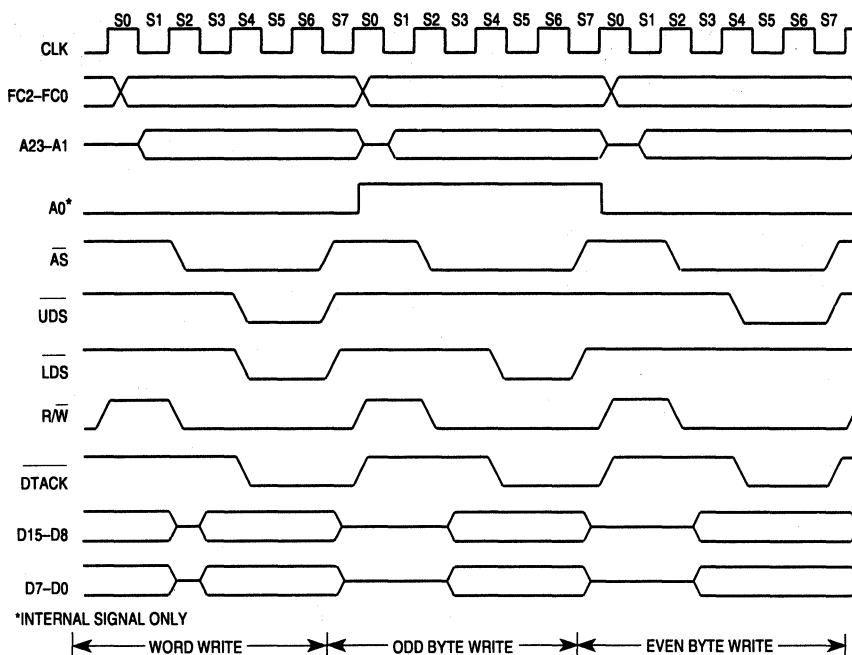


Figure 5-7. Word and Byte Write-Cycle Timing Diagram

The descriptions of the eight states of a write cycle are as follows:

- STATE 0** The write cycle starts in S0. The processor places valid function codes on FC2-FC0 and drives R/W high (if a preceding write cycle has left R/W low).
- STATE 1** Entering S1, the processor drives a valid address on the address bus.
- STATE 2** On the rising edge of S2, the processor asserts $\overline{AS}$ and drives R/W low.
- STATE 3** During S3, the data bus is driven out of the high-impedance state as the data to be written is placed on the bus.
- STATE 4** At the rising edge of S4, the processor asserts $\overline{UDS}$, or $\overline{LDS}$. The processor waits for a cycle termination signal ($\overline{DTACK}$ or $\overline{BERR}$) or $\overline{VPA}$, an M6800 peripheral signal. When $\overline{VPA}$ is asserted during S4, the cycle becomes a peripheral cycle (refer to **Appendix B M6800 Peripheral Interface**). If neither termination signal is asserted before the falling edge at the end of S4, the processor inserts wait states (full clock cycles) until either $\overline{DTACK}$ or $\overline{BERR}$ is asserted.
- STATE 5** During S5, no bus signals are altered.
- STATE 6** During S6, no bus signals are altered.

STATE 7 On the falling edge of the clock entering S7, the processor negates $\overline{AS}$, $\overline{UDS}$, or $\overline{LDS}$. As the clock rises at the end of S7, the processor places the address and data buses in the high-impedance state, and drives R/W high. The device negates $\overline{DTACK}$ or $\overline{BERR}$ at this time.

5.1.3 Read-Modify-Write Cycle.

The read-modify-write cycle performs a read operation, modifies the data in the arithmetic logic unit, and writes the data back to the same address. The address strobe ($\overline{AS}$) remains asserted throughout the entire cycle, making the cycle indivisible. The test and set (TAS) instruction uses this cycle to provide a signaling capability without deadlock between processors in a multiprocessing environment. The TAS instruction (the only instruction that uses the read-modify-write cycle) only operates on bytes. Thus, all read-modify-write cycles are byte operations. The read-modify-write flowchart shown in Figure 5-8 and the timing diagram in Figure 5-9, applies to the MC68000, the MC68HC000, the MC68HC001 (in 16-bit mode), the MC68EC000 (in 16-bit mode), and the MC68010.

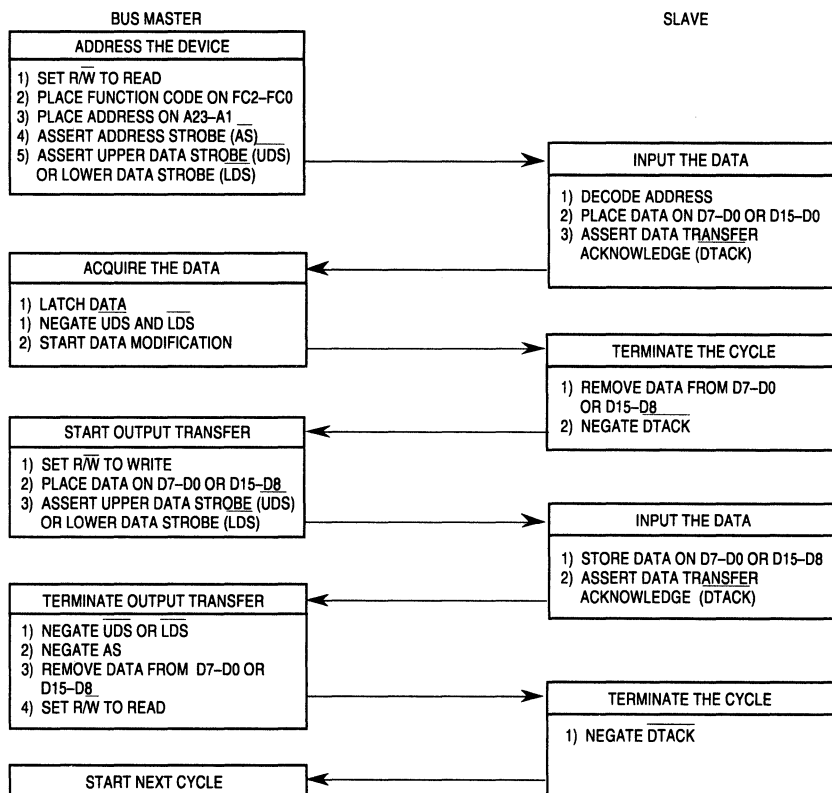


Figure 5-8. Read-Modify-Write Cycle Flowchart

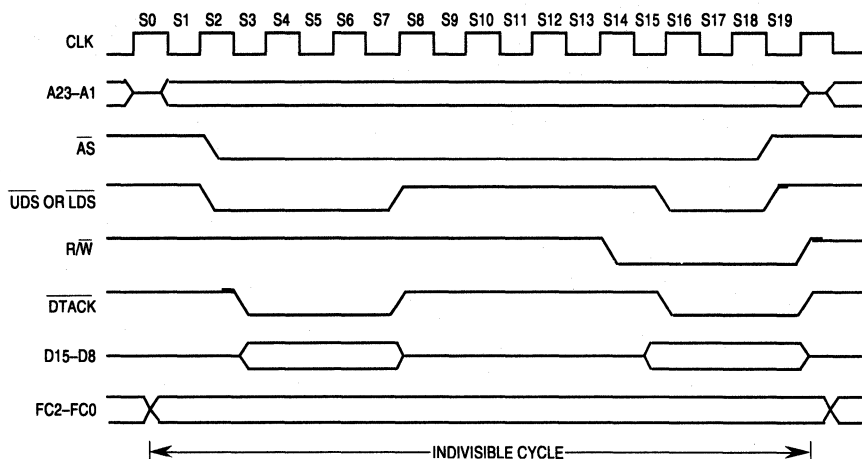


Figure 5-9. Read-Modify-Write Cycle Timing Diagram

The descriptions of the read-modify-write cycle states are as follows:

- STATE 0** The read cycle starts in S0. The processor places valid function codes on FC2-FC0 and drives R/W high to identify a read cycle.
- STATE 1** Entering S1, the processor drives a valid address on the address bus.
- STATE 2** On the rising edge of S2, the processor asserts $\overline{AS}$ and $\overline{UDS}$, or $\overline{LDS}$.
- STATE 3** During S3, no bus signals are altered.
- STATE 4** During S4, the processor waits for a cycle termination signal ($\overline{DTACK}$ or $\overline{BERR}$) or VPA, an M6800 peripheral signal. When VPA is asserted during S4, the cycle becomes a peripheral cycle (refer to **Appendix B M6800 Peripheral Interface**). If neither termination signal is asserted before the falling edge at the end of S4, the processor inserts wait states (full clock cycles) until either $\overline{DTACK}$ or $\overline{BERR}$ is asserted.
- STATE 5** During S5, no bus signals are altered.
- STATE 6** During S6, data from the device are driven onto the data bus.
- STATE 7** On the falling edge of the clock entering S7, the processor accepts data from the device and negates $\overline{UDS}$, and $\overline{LDS}$. The device negates $\overline{DTACK}$ or $\overline{BERR}$ at this time.
- STATES 8-11** The bus signals are unaltered during S8-S11, during which the arithmetic logic unit makes appropriate modifications to the data.

- STATE 12 The write portion of the cycle starts in S12. The valid function codes on FC2–FC0, the address bus lines, $\overline{AS}$, and $R/\overline{W}$ remain unaltered.
- STATE 13 During S13, no bus signals are altered.
- STATE 14 On the rising edge of S14, the processor drives $R/\overline{W}$ low.
- STATE 15 During S15, the data bus is driven out of the high-impedance state as the data to be written are placed on the bus.
- STATE 16 At the rising edge of S16, the processor asserts $\overline{UDS}$ or $\overline{LDS}$. The processor waits for $\overline{DTACK}$ or $\overline{BERR}$ or $\overline{VPA}$, an M6800 peripheral signal. When $\overline{VPA}$ is asserted during S16, the cycle becomes a peripheral cycle (refer to **Appendix B M6800 Peripheral Interface**). If neither termination signal is asserted before the falling edge at the close of S16, the processor inserts wait states (full clock cycles) until either $\overline{DTACK}$ or $\overline{BERR}$ is asserted.
- STATE 17 During S17, no bus signals are altered.
- STATE 18 During S18, no bus signals are altered.
- STATE 19 On the falling edge of the clock entering S19, the processor negates $\overline{AS}$, $\overline{UDS}$, and $\overline{LDS}$. As the clock rises at the end of S19, the processor places the address and data buses in the high-impedance state, and drives $R/\overline{W}$ high. The device negates $\overline{DTACK}$ or $\overline{BERR}$ at this time.

5.1.4 CPU Space Cycle

A CPU space cycle, indicated when the function codes are all high, is a special processor cycle. Bits A16–A19 of the address bus identify eight types of CPU space cycles. Only the interrupt acknowledge cycle, in which A16–A19 are high, applies to all the microprocessors described in this manual. The MC68010 defines an additional type of CPU space cycle, the breakpoint acknowledge cycle, in which A16–A19 are all low. Other configurations of A16–A19 are reserved by Motorola to define other types of CPU cycles used in other M68000 Family microprocessors. Figure 5-10 shows the encoding of CPU space addresses.

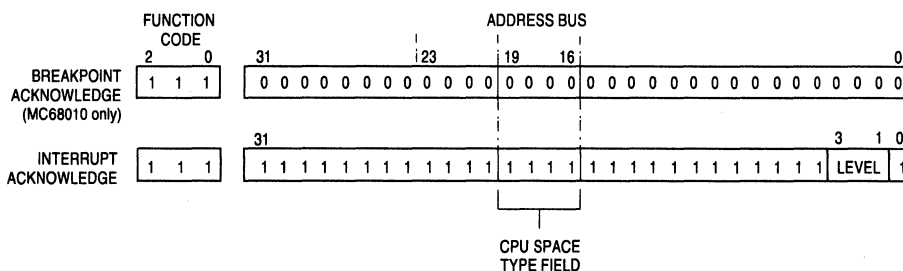
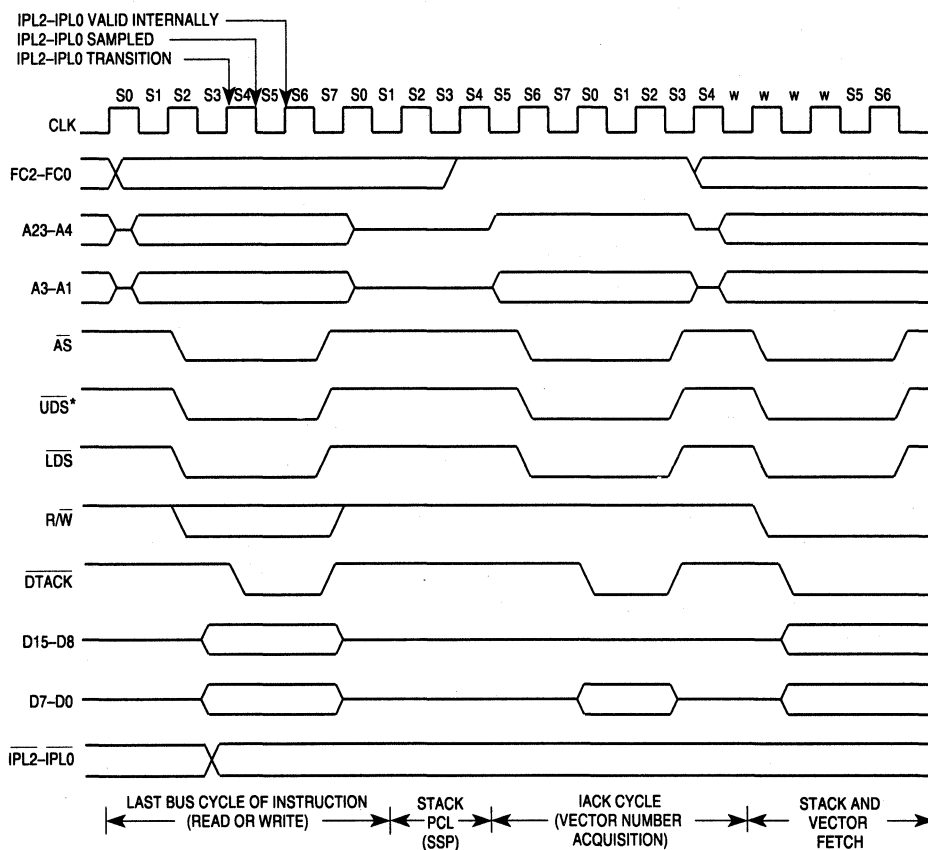


Figure 5-10. CPU Space Address Encoding

The interrupt acknowledge cycle places the level of the interrupt being acknowledged on address bits A3–A1 and drives all other address lines high. The interrupt acknowledge cycle reads a vector number when the interrupting device places a vector number on the data bus and asserts $\overline{DTACK}$ to acknowledge the cycle.

The timing diagram for an interrupt acknowledge cycle is shown in Figure 5-11. Alternately, the interrupt acknowledge cycle can be autovectored. The interrupt acknowledge cycle is the same, except the interrupting device asserts $\overline{VPA}$ instead of $\overline{DTACK}$. For an autovectored interrupt, the vector number used is \$18 plus the interrupt level. This is generated internally by the microprocessor when $\overline{VPA}$ (or $\overline{AVEC}$) is asserted on an interrupt acknowledge cycle. $\overline{DTACK}$ and $\overline{VPA}$ ($\overline{AVEC}$) should never be simultaneously asserted.



* Although a vector number is one byte, both data strobes are asserted due to the microcode used for exception processing. The processor does not recognize anything on data lines D8 through D15 at this time.

Figure 5-11. Interrupt Acknowledge Cycle Timing Diagram

The breakpoint acknowledge cycle is performed by the MC68010 to provide an indication to hardware that a software breakpoint is being executed when the processor executes a breakpoint (BKPT) instruction. The processor neither accepts nor sends data during this cycle, which is otherwise similar to a read cycle. The cycle is terminated by either $\overline{DTACK}$, $\overline{BERR}$, or as an M6800 peripheral cycle when $\overline{VPA}$ is asserted, and the processor continues illegal instruction exception processing. Figure 5-12 illustrates the timing diagram for the breakpoint acknowledge cycle.

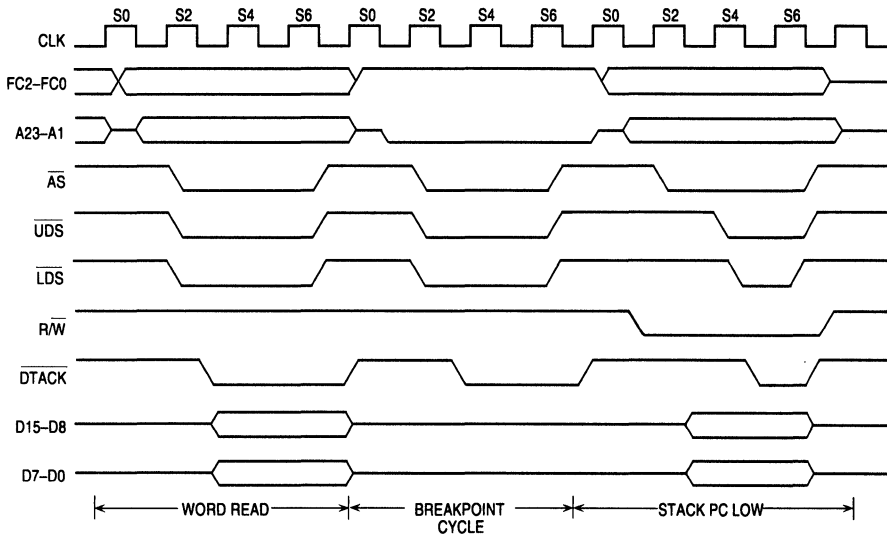


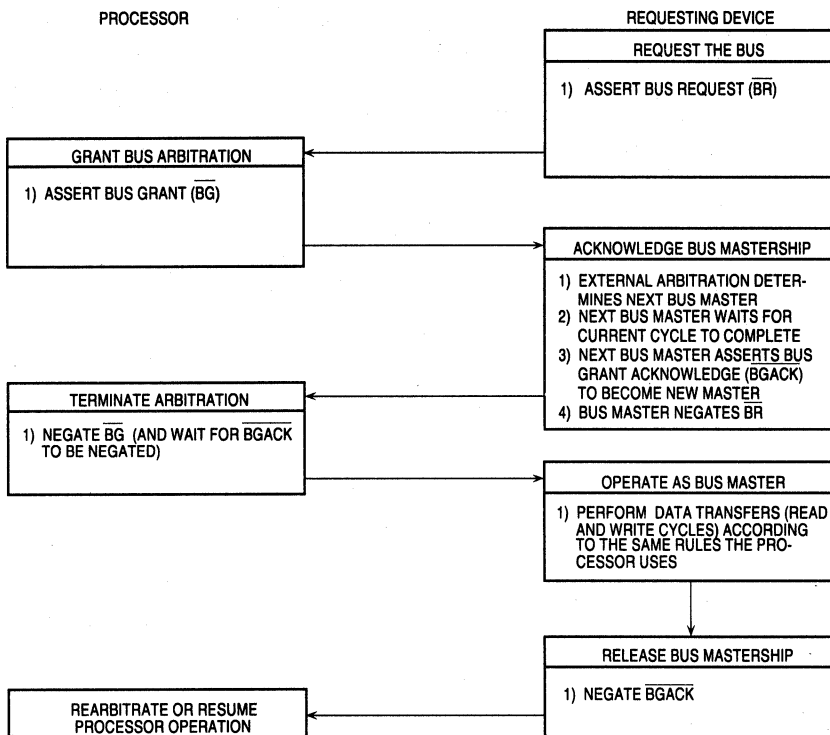
Figure 5-12. Breakpoint Acknowledge Cycle Timing Diagram

5.2 BUS ARBITRATION

Bus arbitration is a technique used by bus master devices to request, to be granted, and to acknowledge bus mastership. Bus arbitration consists of the following:

1. Asserting a bus mastership request
2. Receiving a grant indicating that the bus is available at the end of the current cycle
3. Acknowledging that mastership has been assumed

There are two ways to arbitrate the bus, 3-wire and 2-wire bus arbitration. The MC68000, MC68HC000, MC68EC000, MC68HC001, MC68008, and MC68010 can do 2-wire bus arbitration. The MC68000, MC68HC000, MC68HC001, and MC68010 can do 3-wire bus arbitration. Figures 5-13 and 5-15 show 3-wire bus arbitration and Figures 5-14 and 5-16 show 2-wire bus arbitration. Bus arbitration on all microprocessors, except the 48-pin MC68008 and MC68EC000, $\overline{BGACK}$ must be pulled high for 2-wire bus arbitration.



**Figure 5-13. 3-Wire Bus Arbitration Cycle Flowchart
(Not Applicable to 48-Pin MC68008 or MC68EC000)**

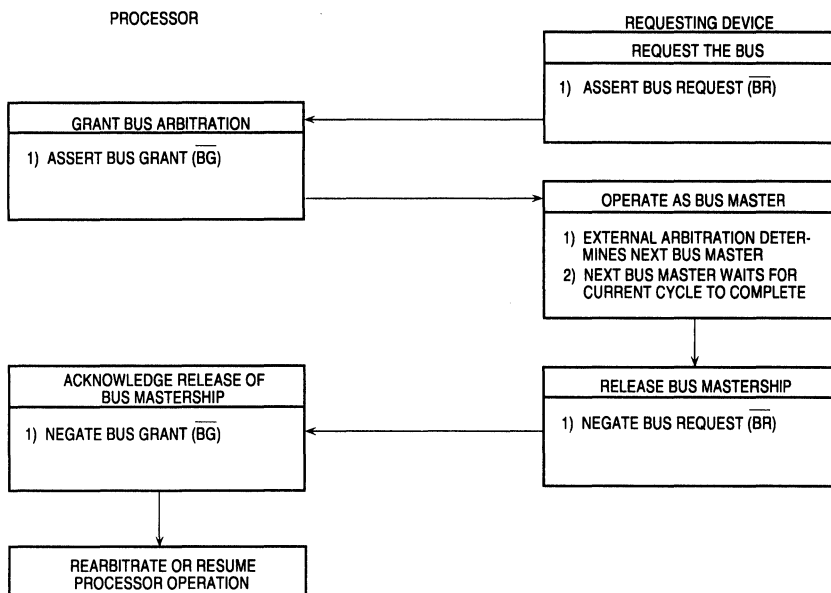
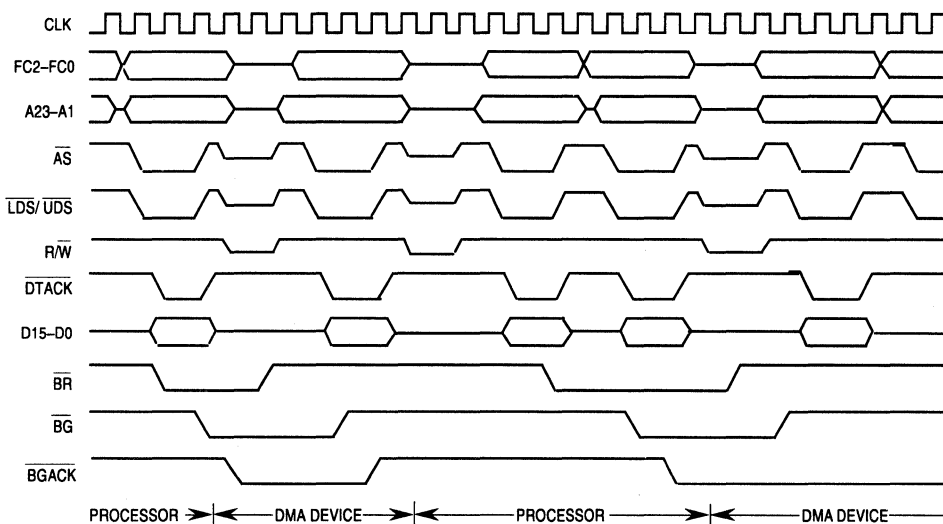


Figure 5-14. 2-Wire Bus Arbitration Cycle Flowchart



**Figure 5-15. 3-Wire Bus Arbitration Timing Diagram
(Not Applicable to 48-Pin MC68008 or MC68EC000)**

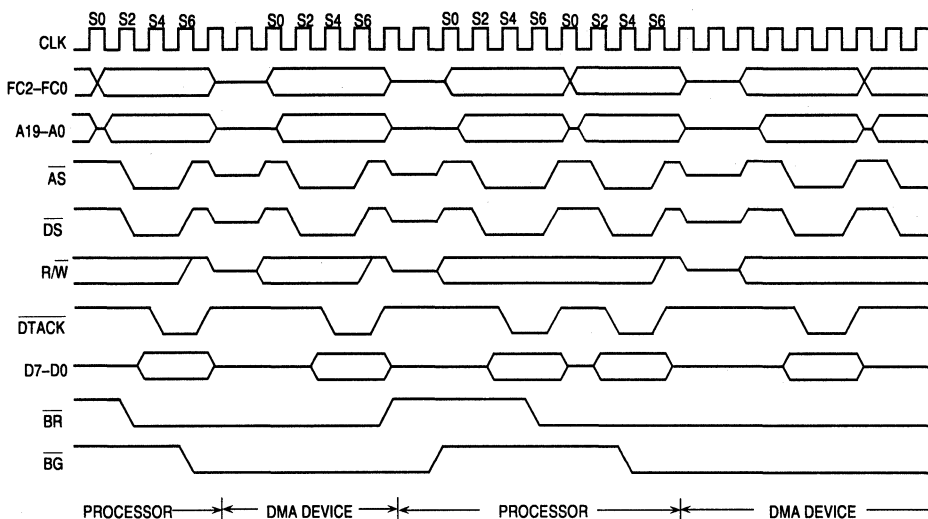


Figure 5-16. 2-Wire Bus Arbitration Timing Diagram

The timing diagram in Figure 5-15 shows that the bus request is negated at the time that an acknowledge is asserted. This type of operation applies to a system consisting of a processor and one other device capable of becoming bus master. In systems having several devices that can be bus masters, bus request lines from these devices can be wire-ORed at the processor, and more than one bus request signal could occur.

The bus grant signal is negated a few clock cycles after the assertion of the bus grant acknowledge signal. However, if bus requests are pending, the processor reasserts bus grant for another request a few clock cycles after bus grant (for the previous request) is negated. In response to this additional assertion of bus grant, external arbitration circuitry selects the next bus master before the current bus master has completed the bus activity.

The timing diagram in Figure 5-15 also applies to a system consisting of a processor and one other device capable of becoming bus master. Since the 48-pin version of the MC68008 and the MC68EC000 does not recognize a bus grant acknowledge signal, this processor does not negate bus grant until the current bus master has completed the bus activity.

5.2.1 Requesting The Bus

External devices capable of becoming bus masters assert $\overline{BR}$ to request the bus. This signal can be wire-ORed (not necessarily constructed from open-collector devices) from any of the devices in the system that can become bus master. The processor, which is at a lower bus priority level than the external devices, relinquishes the bus after it completes the current bus cycle.

The bus grant acknowledge signal on all the processors except the 48-pin MC68008 and MC68EC000 helps to prevent the bus arbitration circuitry from responding to noise on the

bus request signal. When no acknowledge is received before the bus request signal is negated, the processor continues the use of the bus.

5.2.2 Receiving The Bus Grant

The processor asserts $\overline{BG}$ as soon as possible. Normally, this process immediately follows internal synchronization, except when the processor has made an internal decision to execute the next bus cycle but has not yet asserted $\overline{AS}$ for that cycle. In this case, $\overline{BG}$ is delayed until $\overline{AS}$ is asserted to indicate to external devices that a bus cycle is in progress.

$\overline{BG}$ can be routed through a daisy-chained network or through a specific priority-encoded network. Any method of external arbitration that observes the protocol can be used.

5.2.3 Acknowledgment Of Mastership (3-Wire Bus Arbitration Only)

Upon receiving $\overline{BG}$, the requesting device waits until $\overline{AS}$, $\overline{DTACK}$, and $\overline{BGACK}$ are negated before asserting $\overline{BGACK}$. The negation of $\overline{AS}$ indicates that the previous bus master has completed its cycle. (No device is allowed to assume bus mastership while $\overline{AS}$ is asserted.) The negation of $\overline{BGACK}$ indicates that the previous master has released the bus. The negation of $\overline{DTACK}$ indicates that the previous slave has terminated the connection to the previous master. (In some applications, $\overline{DTACK}$ might not be included in this function; general-purpose devices would be connected using $\overline{AS}$ only.) When $\overline{BGACK}$ is asserted, the asserting device is bus master until it negates $\overline{BGACK}$. $\overline{BGACK}$ should not be negated until after the bus cycle(s) is complete. A device relinquishes control of the bus by negating $\overline{BGACK}$.

The bus request from the granted device should be negated after $\overline{BGACK}$ is asserted. If another bus request is pending, $\overline{BG}$ is reasserted within a few clocks, as described in **5.3 Bus Arbitration Control**. The processor does not perform any external bus cycles before reasserting $\overline{BG}$.

5.3 BUS ARBITRATION CONTROL

All asynchronous bus arbitration signals to the processor are synchronized before being used internally. As shown in Figure 5-17, synchronization requires a maximum of one cycle of the system clock, assuming that the asynchronous input setup time (#47, defined in **Section 10 Electrical Characteristic**) has been met. The input asynchronous signal is sampled on the falling edge of the clock and is valid internally after the next falling edge.

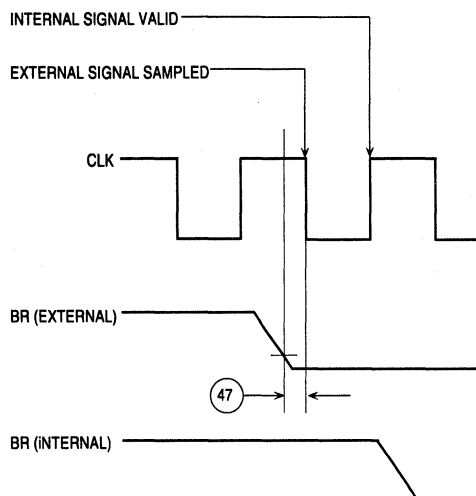


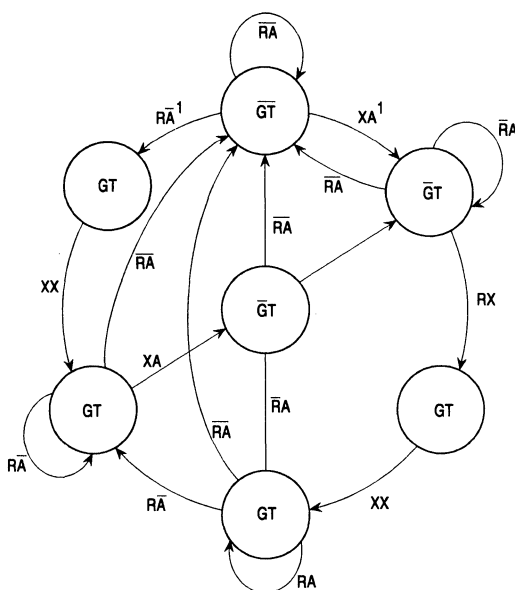
Figure 5-17. External Asynchronous Signal Synchronization

Bus arbitration control is implemented with a finite-state machine. State diagram (a) in Figure 5-18 applies to all processors using 3-wire bus arbitration and state diagram (b) applies to processors using 2-wire bus arbitration, in which $\overline{\text{BGACK}}$ is permanently negated internally or externally. The same finite-state machine is used, but it is effectively a two-state machine because $\overline{\text{BGACK}}$ is always negated.

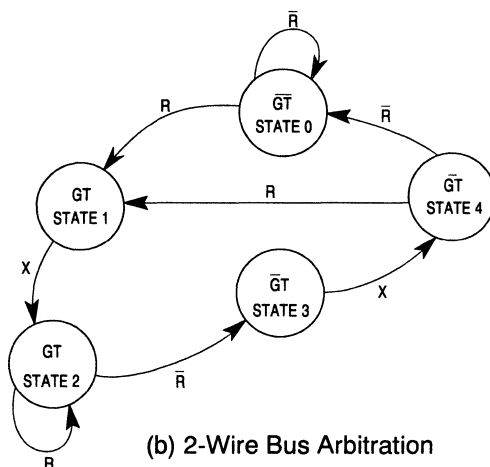
In Figure 5-18, input signals R and A are the internally synchronized versions of $\overline{\text{BR}}$ and $\overline{\text{BGACK}}$. The $\overline{\text{BG}}$ output is shown as G, and the internal three-state control signal is shown as T. If T is true, the address, data, and control buses are placed in the high-impedance state when $\overline{\text{AS}}$ is negated. All signals are shown in positive logic (active high), regardless of their true active voltage level. State changes (valid outputs) occur on the next rising edge of the clock after the internal signal is valid.

A timing diagram of the bus arbitration sequence during a processor bus cycle is shown in Figure 5-19. The bus arbitration timing while the bus is inactive (e.g., the processor is performing internal operations for a multiply instruction) is shown in Figure 5-20.

When a bus request is made after the MPU has begun a bus cycle and before $\overline{\text{AS}}$ has been asserted (S0), the special sequence shown in Figure 5-21 applies. Instead of being asserted on the next rising edge of clock, $\overline{\text{BG}}$ is delayed until the second rising edge following its internal assertion.



(a) 3-Wire Bus Arbitration



(b) 2-Wire Bus Arbitration

R = Bus Request Internal
 A = Bus Grant Acknowledge Internal
 G = Bus Grant
 T = Three-state Control to Bus Control Logic
 X = Don't Care

Notes:

1. State machine will not change if the bus is S0 or S1. Refer to **BUS ARBITRATION CONTROL, 5.2.3.**
2. The address bus will be placed in the high-impedance state if T is asserted and $\overline{AS}$ is negated.

Figure 5-18. Bus Arbitration Unit State Diagrams

Figures 5-19, 5-20, and 5-21 applies to all processors using 3-wire bus arbitration. Figures 5-22, 5-23, and 5-24 applies to all processors using 2-wire bus arbitration.

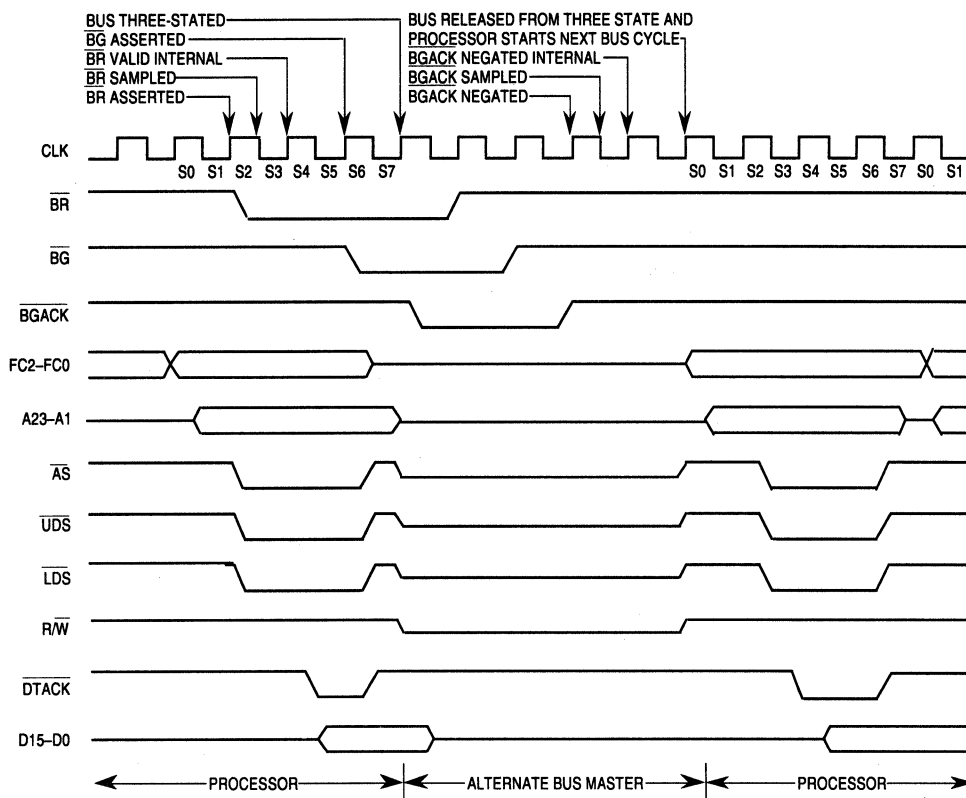


Figure 5-19. 3-Wire Bus Arbitration Timing Diagram—Processor Active

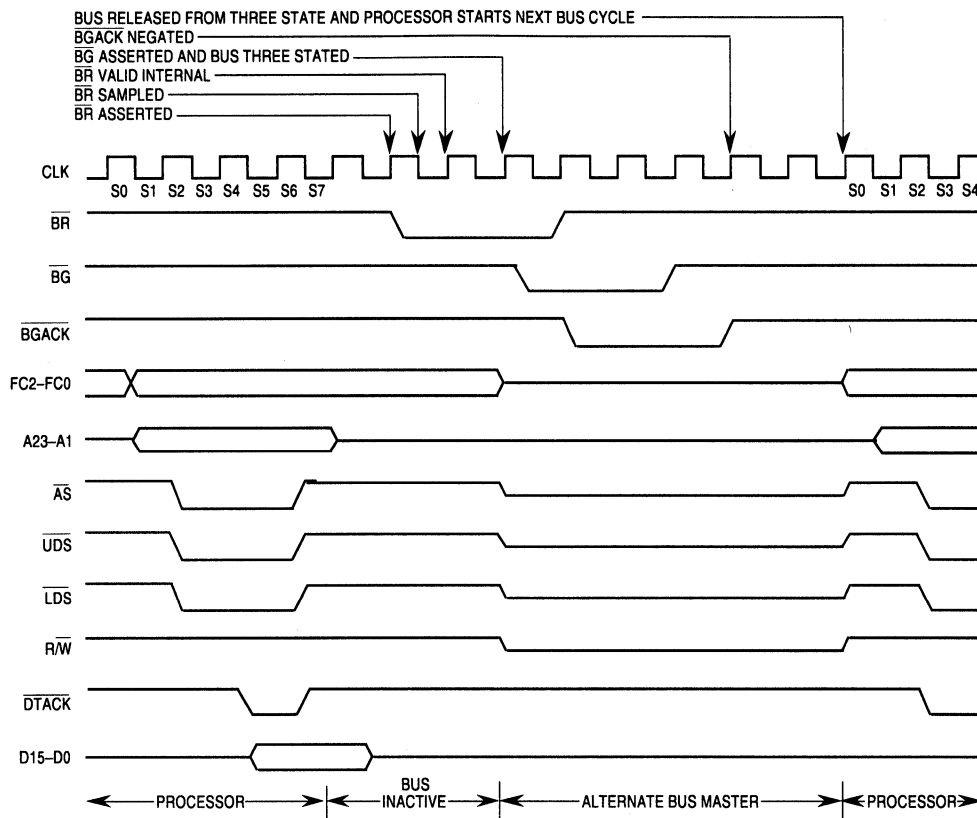


Figure 5-20. 3-Wire Bus Arbitration Timing Diagram—Bus Inactive

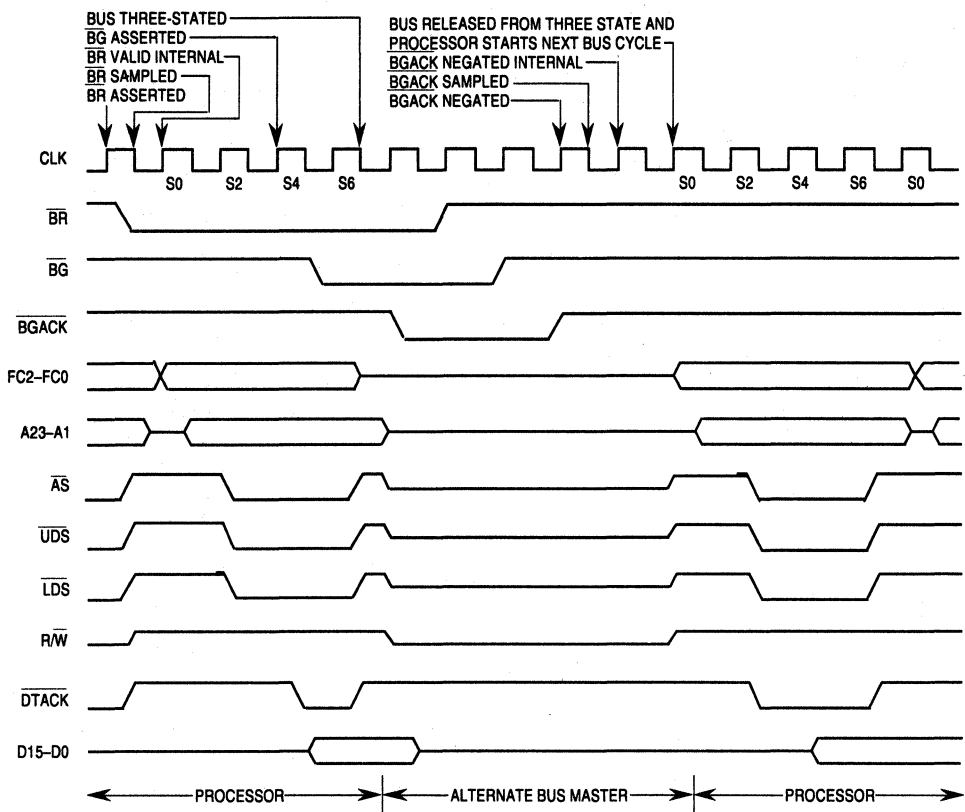


Figure 5-21. 3-Wire Bus Arbitration Timing Diagram—Special Case

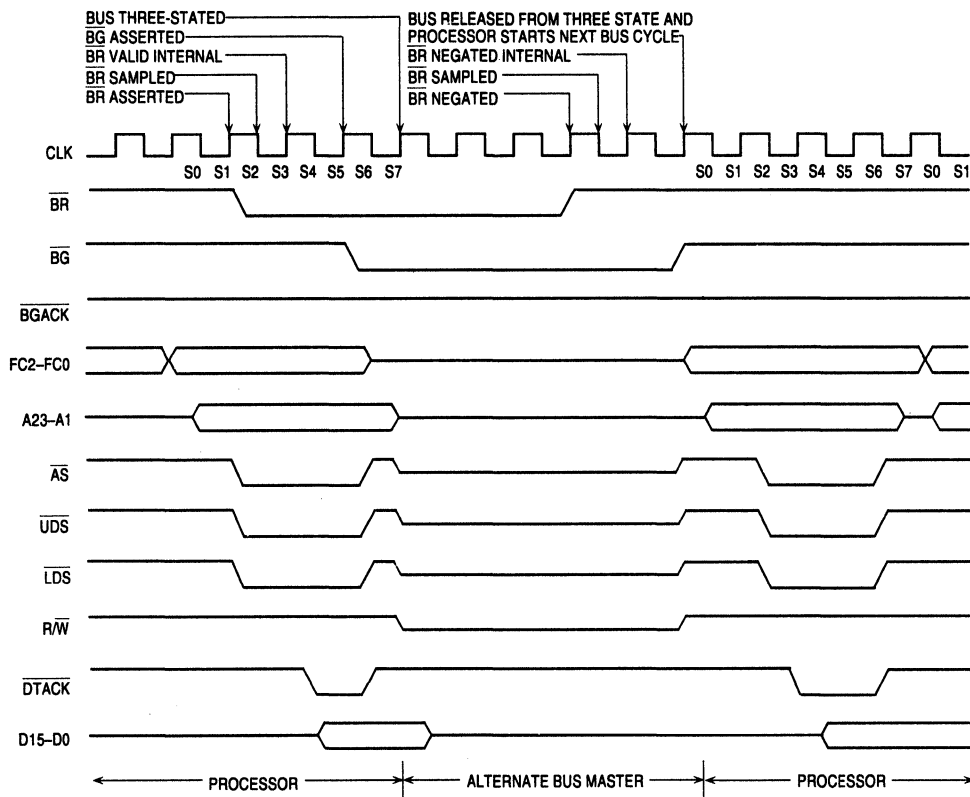


Figure 5-22. 2-Wire Bus Arbitration Timing Diagram—Processor Active

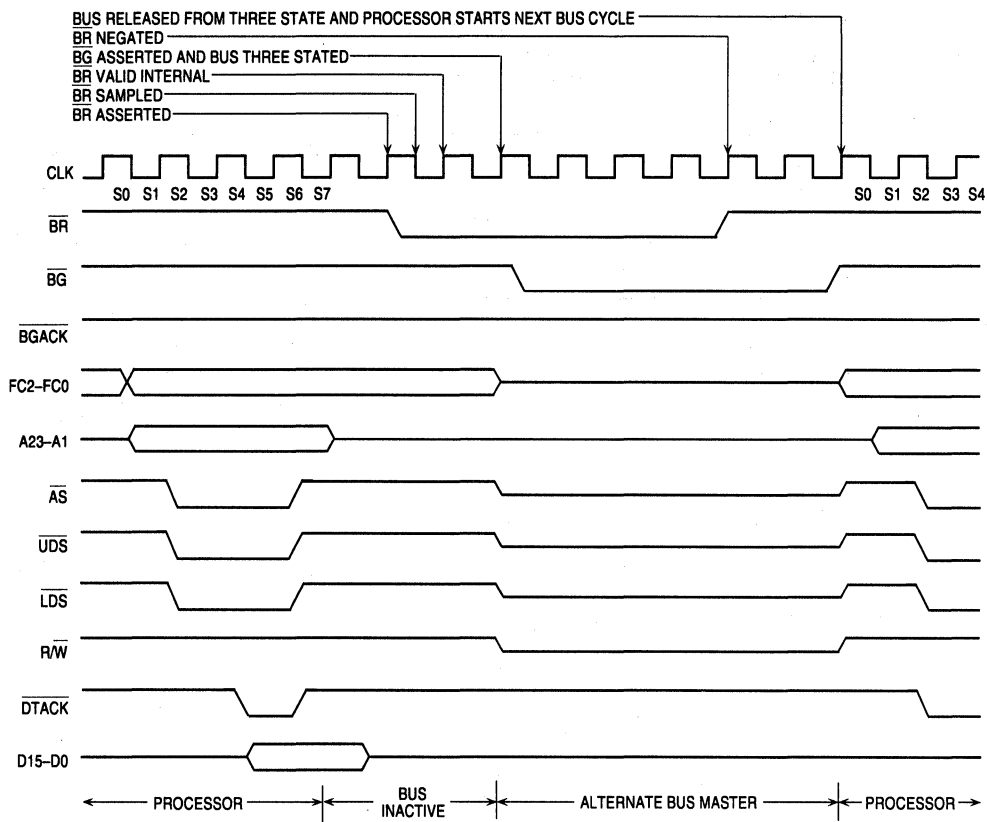


Figure 5-23. 2-Wire Bus Arbitration Timing Diagram—Bus Inactive

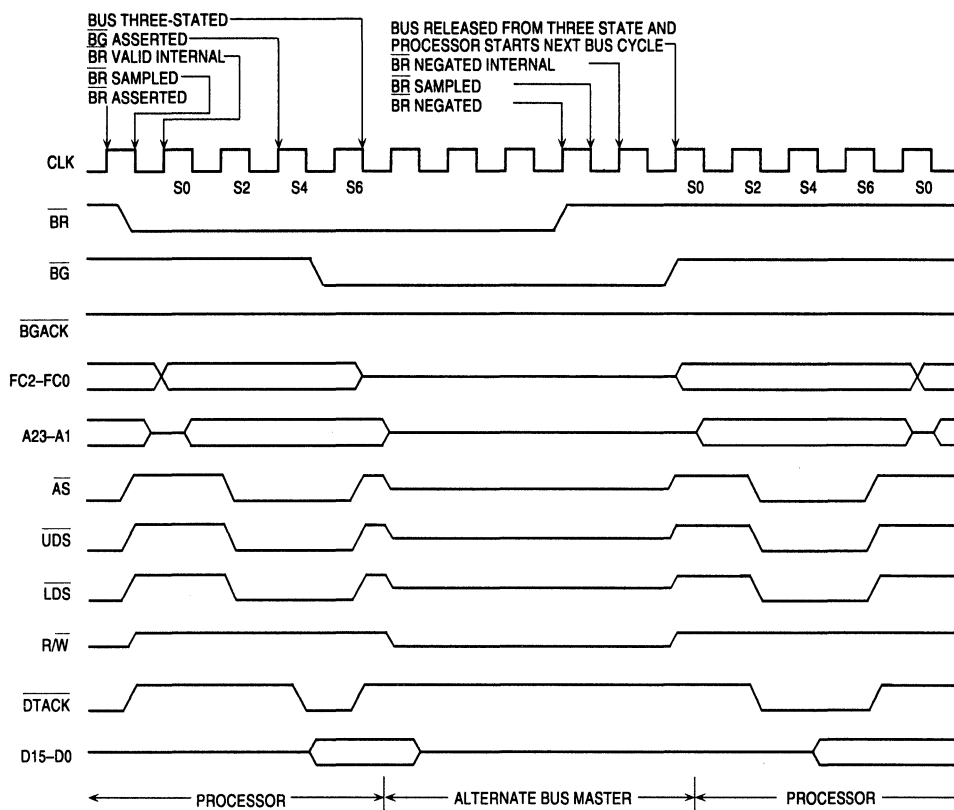


Figure 5-24. 2-Wire Bus Arbitration Timing Diagram—Special Case

5.4. BUS ERROR AND HALT OPERATION

In a bus architecture that requires a handshake from an external device, such as the asynchronous bus used in the M68000 Family, the handshake may not always occur. A bus error input is provided to terminate a bus cycle in error when the expected signal is not asserted. Different systems and different devices within the same system require different maximum-response times. External circuitry can be provided to assert the bus error signal after the appropriate delay following the assertion of address strobe.

In a virtual memory system, the bus error signal can be used to indicate either a page fault or a bus timeout. An external memory management unit asserts bus error when the page that contains the required data is not resident in memory. The processor suspends execution of the current instruction while the page is loaded into memory. The MC68010 pushes enough information on the stack to be able to resume execution of the instruction following return from the bus error exception handler.

The MC68010 also differs from the other microprocessors described in this manual regarding bus errors. The MC68010 can detect a late bus error signal asserted within one clock cycle after the assertion of data transfer acknowledge. When receiving a bus error signal, the processor can either initiate a bus error exception sequence or try running the cycle again.

5.4.1 Bus Error Operation

In all the microprocessors described in this manual, a bus error is recognized when $\overline{DTACK}$ and $\overline{HALT}$ are negated and $\overline{BERR}$ is asserted. In the MC68010, a late bus error is also recognized when $\overline{HALT}$ is negated, and $\overline{DTACK}$ and $\overline{BERR}$ are asserted within one clock cycle.

When the bus error condition is recognized, the current bus cycle is terminated in S9 for a read cycle, a write cycle, or the read portion of a read-modify-write cycle. For the write portion of a read-modify-write cycle, the current bus cycle is terminated in S21. As long as $\overline{BERR}$ remains asserted, the data and address buses are in the high-impedance state. Figure 5-25 shows the timing for the normal bus error, and Figure 5-26 shows the timing for the MC68010 late bus error.

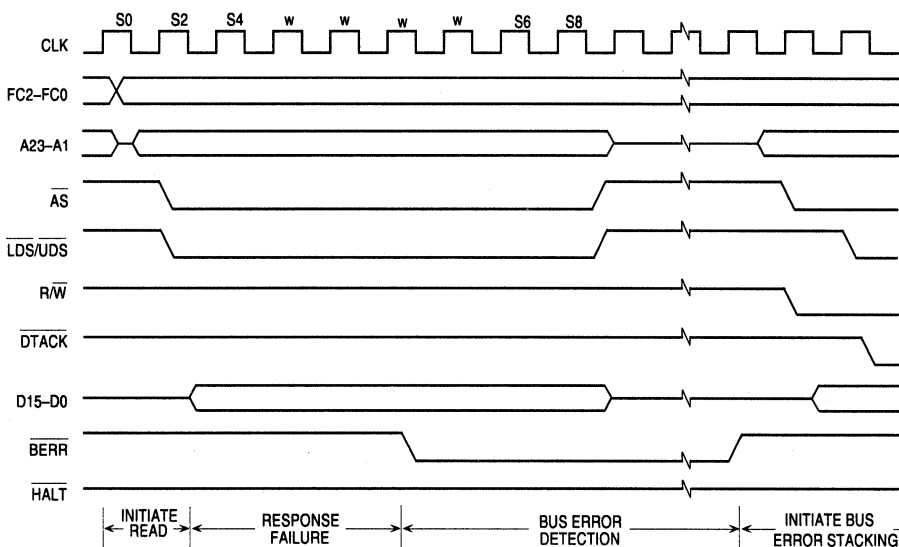


Figure 5-25. Bus Error Timing Diagram

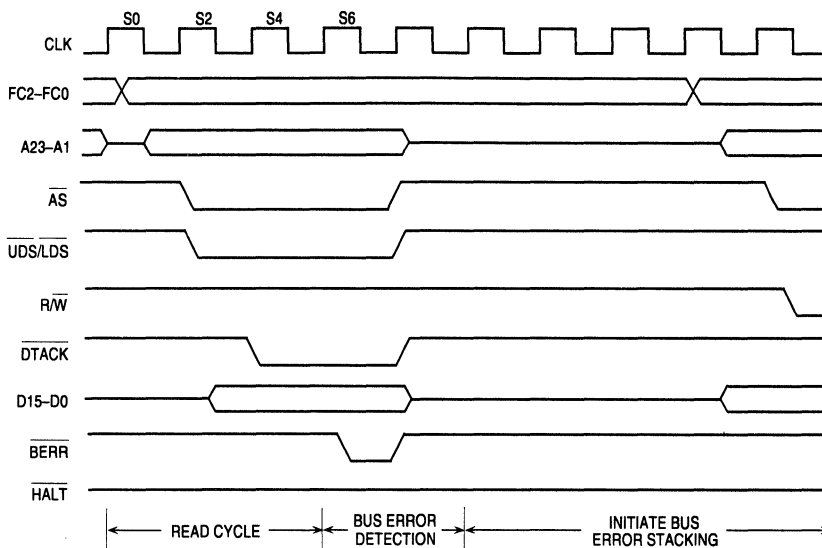


Figure 5-26. Delayed Bus Error Timing Diagram (MC68010)

After the aborted bus cycle is terminated and $\overline{\text{BERR}}$ is negated, the processor enters exception processing for the bus error exception. During the exception processing sequence, the following information is placed on the supervisor stack:

1. Status register
2. Program counter (two words, which may be up to five words past the instruction being executed)
3. Error information

The first two items are identical to the information stacked by any other exception. The error information differs for the MC68010. The MC68000, MC68HC000, MC68HC001, MC68EC000, and MC68008 stack bus error information to help determine and to correct the error. The MC68010 stacks the frame format and the vector offset followed by 22 words of internal register information. The return from exception (RTE) instruction restores the internal register information so that the MC68010 can continue execution of the instruction after the error handler routine completes.

After the processor has placed the required information on the stack, the bus error exception vector is read from vector table entry 2 (offset \$08) and placed in the program counter. The processor resumes execution at the address in the vector, which is the first instruction in the bus error handler routine.

NOTE

In the MC68010, if a read-modify-write operation terminates in a bus error, the processor reruns the entire read-modify-write operation when the RTE instruction at the end of the bus error handler returns control to the instruction in error. The processor reruns the entire operation whether the error occurred during the read or write portion.

5.4.2 Retrying The Bus Cycle

The assertion of the bus error signal during a bus cycle in which $\overline{\text{HALT}}$ is also asserted by an external device initiates a retry operation. Figure 5-27 is a timing diagram of the retry operation. The delayed $\overline{\text{BERR}}$ signal in the MC68010 also initiates a retry operation when $\overline{\text{HALT}}$ is asserted by an external device. Figure 5-28 shows the timing of the delayed operation.

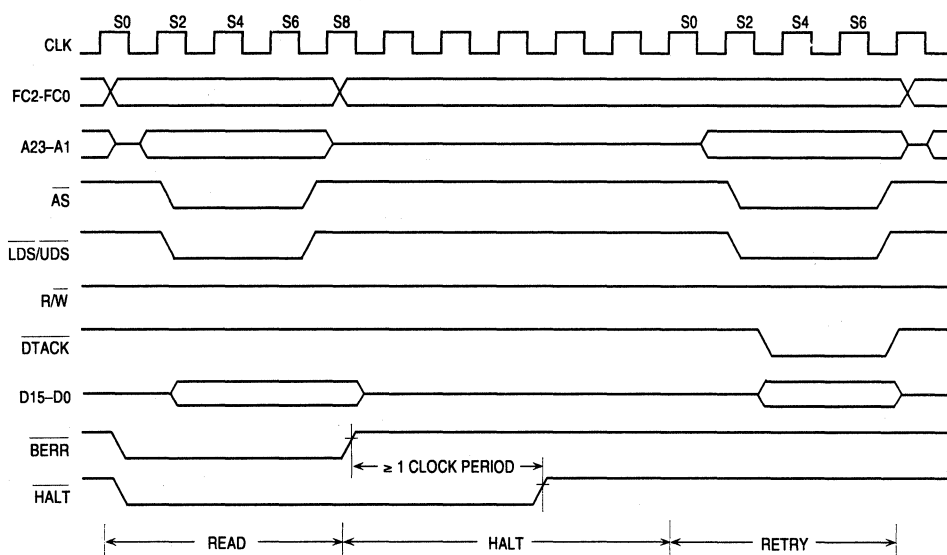


Figure 5-27. Retry Bus Cycle Timing Diagram

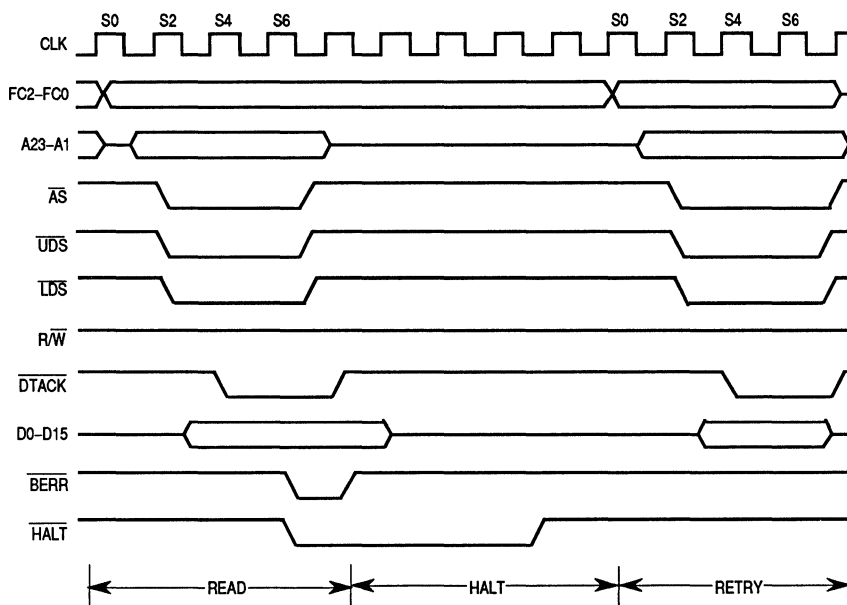


Figure 5-28. Delayed Retry Bus Cycle Timing Diagram

The processor terminates the bus cycle, then puts the address and data lines in the high-impedance state. The processor remains in this state until $\overline{\text{HALT}}$ is negated. Then the processor retries the preceding cycle using the same function codes, address, and data (for a write operation). $\overline{\text{BERR}}$ should be negated at least one clock cycle before $\overline{\text{HALT}}$ is negated.

NOTE

To guarantee that the entire read-modify-write cycle runs correctly and that the write portion of the operation is performed without negating the address strobe, the processor does not retry a read-modify-write cycle. When a bus error occurs during a read-modify-write operation, a bus error operation is performed whether or not $\overline{\text{HALT}}$ is asserted.

5.4.3 Halt Operation ($\overline{\text{HALT}}$)

$\overline{\text{HALT}}$ performs a halt/run/single-step operation similar to the halt operation of an MC68000. When $\overline{\text{HALT}}$ is asserted by an external device, the processor halts and remains halted as long as the signal remains asserted, as shown in Figure 5-29.

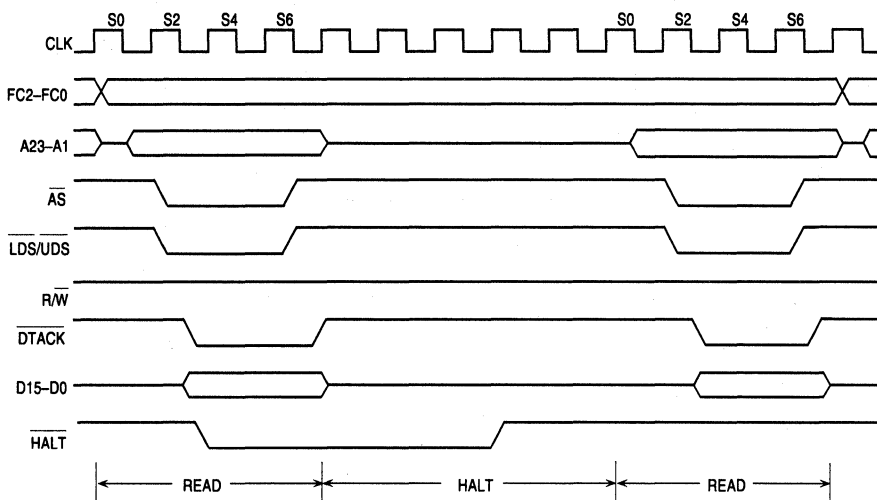


Figure 5-29. Halt Operation Timing Diagram

While the processor is halted, the address bus and the data bus signals are placed in the high-impedance state. Bus arbitration is performed as usual. Should a bus error occur while $\overline{\text{HALT}}$ is asserted, the processor performs the retry operation previously described.

The single-step mode is derived from correctly timed transitions of $\overline{\text{HALT}}$. $\overline{\text{HALT}}$ is negated to allow the processor to begin a bus cycle, then asserted to enter the halt mode when the cycle completes. The single-step mode proceeds through a program one bus cycle at a time for debugging purposes. The halt operation and the hardware trace capability allow tracing of either bus cycles or instructions one at a time. These capabilities and a software debugging package provide total debugging flexibility.

5.4.4 Double Bus Fault

When a bus error exception occurs, the processor begins exception processing by stacking information on the supervisor stack. If another bus error occurs during exception processing (i.e., before execution of another instruction begins) the processor halts and asserts $\overline{\text{HALT}}$. This is called a double bus fault. Only an external reset operation can restart a processor halted due to a double bus fault.

A retry operation does not initiate exception processing; a bus error during a retry operation does not cause a double bus fault. The processor can continue to retry a bus cycle indefinitely if external hardware requests.

A double bus fault occurs during a reset operation when a bus error occurs while the processor is reading the vector table (before the first instruction is executed). The reset operation is described in the following paragraph.

5.5 RESET OPERATION

RESET is asserted externally for the initial processor reset. Subsequently, the signal can be asserted either externally or internally (executing a RESET instruction). For proper external reset operation, **HALT** must also be asserted.

When **RESET** and **HALT** are driven by an external device, the entire system, including the processor, is reset. Resetting the processor initializes the internal state. The processor reads the reset vector table entry (address \$00000) and loads the contents into the supervisor stack pointer (SSP). Next, the processor loads the contents of address \$00004 (vector table entry 1) into the program counter. Then the processor initializes the interrupt level in the status register to a value of seven. In the MC68010, the processor also clears the vector base register to \$00000. No other register is affected by the reset sequence. Figure 5-30 shows the timing of the reset operation.

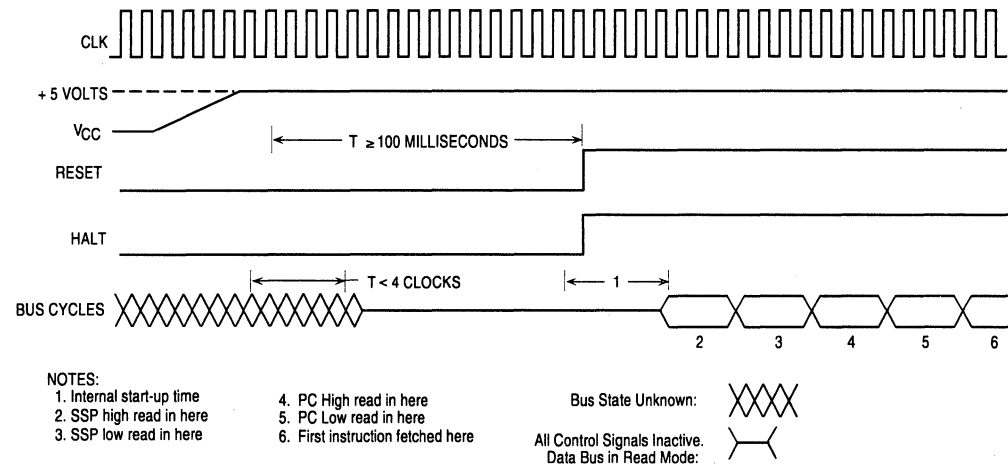


Figure 5-30. Reset Operation Timing Diagram

The RESET instruction causes the processor to assert **RESET** for 124 clock periods to reset the external devices of the system. The internal state of the processor is not affected. Neither the status register nor any of the internal registers is affected by an internal reset operation. All external devices in the system should be reset at the completion of the RESET instruction.

For the initial reset, **RESET** and **HALT** must be asserted for at least 100 ms. For a subsequent external reset, asserting these signals for 10 clock cycles or longer resets the processor. However, an external reset signal that is asserted while the processor is executing a reset instruction is ignored. Since the processor asserts the **RESET** signal for 124 clock cycles during execution of a reset instruction, an external reset should assert **RESET** for at least 132 clock periods.

5.6 THE RELATIONSHIP OF $\overline{DTACK}$, $\overline{BERR}$, AND $\overline{HALT}$

To properly control termination of a bus cycle for a retry or a bus error condition, $\overline{DTACK}$, $\overline{BERR}$, and $\overline{HALT}$ should be asserted and negated on the rising edge of the processor clock. This relationship assures that when two signals are asserted simultaneously, the required setup time (specification #47, **Section 9 Electrical Characteristics**) for both of them is met during the same bus state. External circuitry should be designed to incorporate this precaution. A related specification, #48, can be ignored when $\overline{DTACK}$, $\overline{BERR}$, and $\overline{HALT}$ are asserted and negated on the rising edge of the processor clock.

The possible bus cycle termination can be summarized as follows (case numbers refer to Table 5-5).

- Normal Termination: $\overline{DTACK}$ is asserted. $\overline{BERR}$ and $\overline{HALT}$ remain negated (case 1).
- Halt Termination: $\overline{HALT}$ is asserted coincident with or preceding $\overline{DTACK}$, and $\overline{BERR}$ remains negated (case 2).
- Bus Error Termination: $\overline{BERR}$ is asserted in lieu of, coincident with, or preceding $\overline{DTACK}$ (case 3). In the MC68010, the late bus error also, $\overline{BERR}$ is asserted following $\overline{DTACK}$ (case 4). $\overline{HALT}$ remains negated and $\overline{BERR}$ is negated coincident with or after $\overline{DTACK}$.
- Retry Termination: $\overline{HALT}$ and $\overline{BERR}$ asserted in lieu of, coincident with, or before $\overline{DTACK}$ (case 5). In the MC68010, the late retry also, $\overline{BERR}$ and $\overline{HALT}$ are asserted following $\overline{DTACK}$ (case 6). $\overline{BERR}$ is negated coincident with or after $\overline{DTACK}$. $\overline{HALT}$ must be held at least one cycle after $\overline{BERR}$.

Table 5-1 shows the details of the resulting bus cycle termination in the M68000 microprocessors for various combinations of signal sequences.

Table 5-1. $\overline{DTACK}$, $\overline{BERR}$, and $\overline{HALT}$ Assertion Results

Case No.	Control Signal	Asserted on Rising Edge of State		MC68000/MC68HC000/001 EC000/MC68008 Results	MC68010 Results
		N	N+2		
1	$\overline{DTACK}$ $\overline{BERR}$ $\overline{HALT}$	A NA NA	S NA X	Normal cycle terminate and continue.	Normal cycle terminate and continue.
2	$\overline{DTACK}$ $\overline{BERR}$ $\overline{HALT}$	A NA A/S	S NA S	Normal cycle terminate and halt. Continue when $\overline{HALT}$ negated.	Normal cycle terminate and halt. Continue when $\overline{HALT}$ negated.
3	$\overline{DTACK}$ $\overline{BERR}$ $\overline{HALT}$	X A NA	X S NA	Terminate and take bus error trap.	Terminate and take bus error trap.
4	$\overline{DTACK}$ $\overline{BERR}$ $\overline{HALT}$	A NA NA	S A NA	Normal cycle terminate and continue.	Terminate and take bus error trap.
5	$\overline{DTACK}$ $\overline{BERR}$ $\overline{HALT}$	X A A/S	X S S	Terminate and retry when $\overline{HALT}$ removed.	Terminate and retry when $\overline{HALT}$ removed.
6	$\overline{DTACK}$ $\overline{BERR}$ $\overline{HALT}$	A NA NA	S A A	Normal cycle terminate and continue.	Terminate and retry when $\overline{HALT}$ removed.

LEGEND:

- N — The number of the current even bus state (e.g., S4, S6, etc.)
- A — Signal asserted in this bus state
- NA — Signal not asserted in this bus state
- X — Don't care
- S — Signal asserted in preceding bus state and remains asserted in this state

NOTE: All operations are subject to relevant setup and hold times.

The negation of $\overline{BERR}$ and $\overline{HALT}$ under several conditions is shown in Table 5-6. ($\overline{DTACK}$ is assumed to be negated normally in all cases; for reliable operation, both $\overline{DTACK}$ and $\overline{BERR}$ should be negated when address strobe is negated).

EXAMPLE A:

A system uses a watchdog timer to terminate accesses to unused address space. The timer asserts $\overline{BERR}$ after timeout (case 3).

EXAMPLE B:

A system uses error detection on random-access memory (RAM) contents. The system designer may:

1. Delay $\overline{DTACK}$ until the data is verified. If data is invalid, return $\overline{BERR}$ and $\overline{HALT}$ simultaneously to retry the error cycle (case 5).
2. Delay $\overline{DTACK}$ until the data is verified. If data is invalid, return $\overline{BERR}$ at the same time as $\overline{DTACK}$ (case 3).
3. For an MC68010, return $\overline{DTACK}$ before data verification. If data is invalid, assert $\overline{BERR}$ and $\overline{HALT}$ to retry the error cycle (case 6).

4. For an MC68010, return $\overline{DTACK}$ before data verification. If data is invalid, assert $\overline{BERR}$ on the next clock cycle (case 4).

Table 5-6. $\overline{BERR}$ and $\overline{HALT}$ Negation Results

Conditions of Termination in Table 4-4	Control Signal	Negated on Rising Edge of State			Results—Next Cycle
		N		N+2	
Bus Error	$\overline{BERR}$ $\overline{HALT}$	• •	or or	• •	Takes bus error trap.
Rerun	$\overline{BERR}$ $\overline{HALT}$	• •	or	•	Illegal sequence; usually traps to vector number 0.
Rerun	$\overline{BERR}$ $\overline{HALT}$	•		•	Reruns the bus cycle.
Normal	$\overline{BERR}$ $\overline{HALT}$	• •	or	•	May lengthen next cycle.
Normal	$\overline{BERR}$ $\overline{HALT}$	•	or	• none	If next cycle is started, it will be terminated as a bus error.

• = Signal is negated in this bus state.

5.7 ASYNCHRONOUS OPERATION

To achieve clock frequency independence at a system level, the bus can be operated in an asynchronous manner. Asynchronous bus operation uses the bus handshake signals to control the transfer of data. The handshake signals are $\overline{AS}$, $\overline{UDS}$, $\overline{LDS}$, $\overline{DS}$ (MC68008 only), $\overline{DTACK}$, $\overline{BERR}$, $\overline{HALT}$, $\overline{AVEC}$ (MC68EC000 only), and $\overline{VPA}$ (only for M6800 peripheral cycles). $\overline{AS}$ indicates the start of the bus cycle, and $\overline{UDS}$, $\overline{LDS}$, and $\overline{DS}$ signal valid data for a write cycle. After placing the requested data on the data bus (read cycle) or latching the data (write cycle), the slave device (memory or peripheral) asserts $\overline{DTACK}$ to terminate the bus cycle. If no device responds or if the access is invalid, external control logic asserts $\overline{BERR}$, or $\overline{BERR}$ and $\overline{HALT}$, to abort or retry the cycle. Figure 5-31 shows the use of the bus handshake signals in a fully asynchronous read cycle. Figure 5-32 shows a fully asynchronous write cycle.

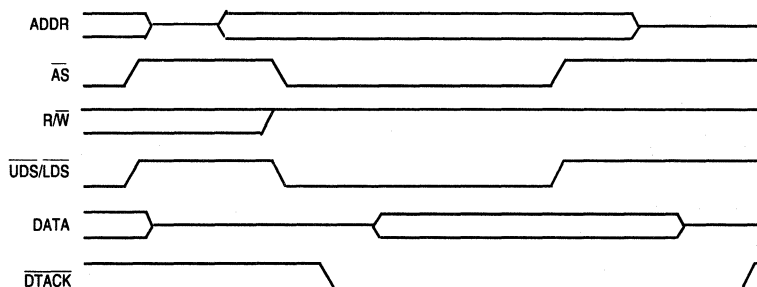


Figure 5-31. Fully Asynchronous Read Cycle

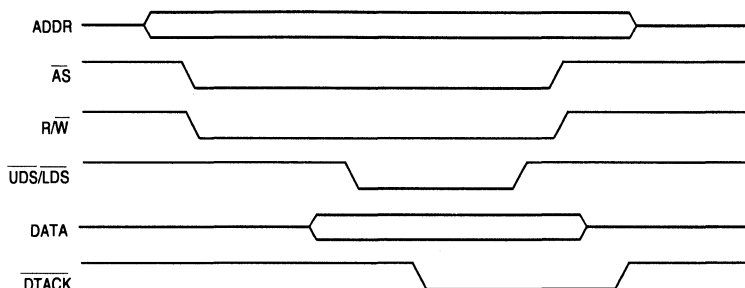


Figure 5-32. Fully Asynchronous Write Cycle

In the asynchronous mode, the accessed device operates independently of the frequency and phase of the system clock. For example, the MC68681 dual universal asynchronous receiver/transmitter (DUART) does not require any clock-related information from the bus master during a bus transfer. Asynchronous devices are designed to operate correctly with processors at any clock frequency when relevant timing requirements are observed.

A device can use a clock at the same frequency as the system clock (e.g., 8, 10, or 12.5, 16, and 20MHz), but without a defined phase relationship to the system clock. This mode of operation is pseudo-asynchronous; it increases performance by observing timing parameters related to the system clock frequency without being completely synchronous with that clock. A memory array designed to operate with a particular frequency processor but not driven by the processor clock is a common example of a pseudo-asynchronous device.

The designer of a fully asynchronous system can make no assumptions about address setup time, which could be used to improve performance. With the system clock frequency known, the slave device can be designed to decode the address bus before recognizing an address strobe. Parameter #11 (refer to **Section 10 Electrical Characteristics**) specifies the minimum time before address strobe during which the address is valid.

In a pseudo-asynchronous system, timing specifications allow $\overline{DTACK}$ to be asserted for a read cycle before the data from a slave device is valid. The length of time that $\overline{DTACK}$ may precede data is specified as parameter #31. This parameter must be met to ensure the validity of the data latched into the processor. No maximum time is specified from the assertion of $\overline{AS}$ to the assertion of $\overline{DTACK}$. During this unlimited time, the processor inserts wait cycles in one-clock-period increments until $\overline{DTACK}$ is recognized. Figure 5-33 shows the important timing parameters for a pseudo-asynchronous read cycle.

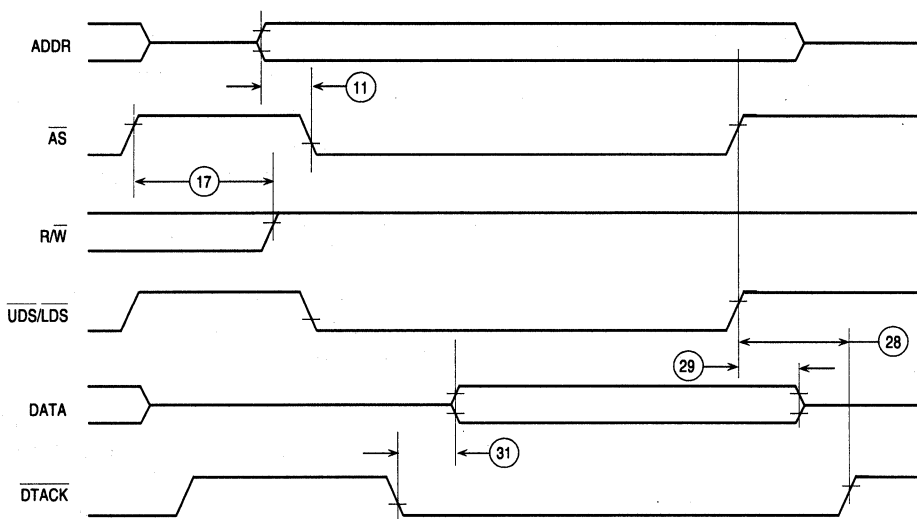


Figure 5-33. Pseudo-Asynchronous Read Cycle

During a write cycle, after the processor asserts $\overline{AS}$ but before driving the data bus, the processor drives $\overline{R/W}$ low. Parameter #55 specifies the minimum time between the transition of $\overline{R/W}$ and the driving of the data bus, which is effectively the maximum turnoff time for any device driving the data bus.

After the processor places valid data on the bus, it asserts the data strobe signal(s). A data setup time, similar to the address setup time previously discussed, can be used to improve performance. Parameter #29 is the minimum time a slave device can accept valid data before recognizing a data strobe. The slave device asserts $\overline{DTACK}$ after it accepts the data. Parameter #25 is the minimum time after negation of the strobes during which the valid data remains on the address bus. Parameter #28 is the maximum time between the negation of the strobes by the processor and the negation of $\overline{DTACK}$ by the slave device. If $\overline{DTACK}$ remains asserted past the time specified by parameter #28, the processor may recognize it as being asserted early in the next bus cycle and may terminate that cycle prematurely. Figure 5-34 shows the important timing specifications for a pseudo-asynchronous write cycle.

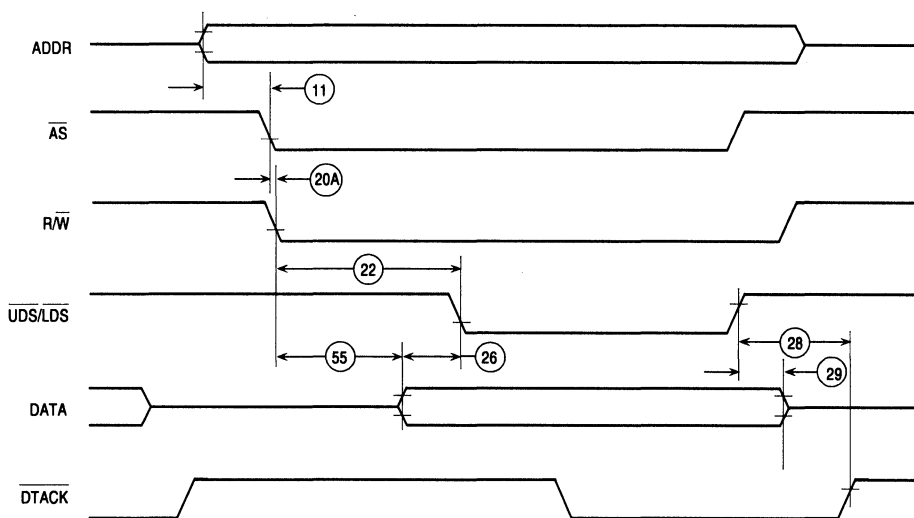


Figure 5-34. Pseudo-Asynchronous Write Cycle

In the MC68010, the $\overline{BERR}$ signal can be delayed after the assertion of $\overline{DTACK}$. Specification #48 is the maximum time between assertion of $\overline{DTACK}$ and assertion of $\overline{BERR}$. If this maximum delay is exceeded, operation of the processor may be erratic.

5.8 SYNCHRONOUS OPERATION

In some systems, external devices use the system clock to generate $\overline{DTACK}$ and other asynchronous input signals. This synchronous operation provides a closely coupled design with maximum performance, appropriate for frequently accessed parts of the system. For example, memory can operate in the synchronous mode, but peripheral devices operate asynchronously. For a synchronous device, the designer uses explicit timing information shown in **Section 10 Electrical Characteristics**. These specifications define the state of all bus signals relative to a specific state of the processor clock.

The standard M68000 bus cycle consists of four clock periods (eight bus cycle states) and, optionally, an integral number of clock cycles inserted as wait states. Wait states are inserted as required to allow sufficient response time for the external device. The following state-by-state description of the bus cycle differs from those descriptions in **5.1.1 READ CYCLE** and **5.1.2 WRITE CYCLE** by including information about the important timing parameters that apply in the bus cycle states.

STATE 0 The bus cycle starts in S0, during which the clock is high. At the rising edge of S0, the function code for the access is driven externally. Parameter #6A defines the delay from this rising edge until the function codes are valid. Also, the $\overline{R/\overline{W}}$ signal is driven high; parameter #18 defines the delay from the same rising edge to the transition of $\overline{R/\overline{W}}$. The minimum value for parameter #18 applies to a read cycle preceded by a write cycle; this value

is the maximum hold time for a low on $\overline{R/\overline{W}}$ beyond the initiation of the read cycle.

- STATE 1 Entering S1, a low period of the clock, the address of the accessed device is driven externally with an assertion delay defined by parameter #6.
- STATE 2 On the rising edge of S2, a high period of the clock, $\overline{AS}$ is asserted. During a read cycle, $\overline{UDS}$, $\overline{LDS}$, and/or $\overline{DS}$ is also asserted at this time. Parameter #9 defines the assertion delay for these signals. For a write cycle, the $\overline{R/\overline{W}}$ signal is driven low with a delay defined by parameter #20.
- STATE 3 On the falling edge of the clock entering S3, the data bus is driven out of the high-impedance state with the data being written to the accessed device (in a write cycle). Parameter #23 specifies the data assertion delay. In a read cycle, no signal is altered in S3.

5

- STATE 4 Entering the high clock period of S4, $\overline{UDS}$, $\overline{LDS}$, and/or $\overline{DS}$ is asserted (during a write cycle) on the rising edge of the clock. As in S2 for a read cycle, parameter #9 defines the assertion delay from the rising edge of S4 for $\overline{UDS}$, $\overline{LDS}$, and/or $\overline{DS}$. In a read cycle, no signal is altered by the processor during S4.

Until the falling edge of the clock at the end of S4 (beginning of S5), no response from any external device except $\overline{RESET}$ is acknowledged by the processor. If either $\overline{DTACK}$ or $\overline{BERR}$ is asserted before the falling edge of S4 and satisfies the input setup time defined by parameter #47, the processor enters S5 and the bus cycle continues. If either $\overline{DTACK}$ or $\overline{BERR}$ is asserted but without meeting the setup time defined by parameter #47, the processor may recognize the signal and continue the bus cycle; the result is unpredictable. If neither $\overline{DTACK}$ nor $\overline{BERR}$ is asserted before the next rise of clock, the bus cycle remains in S4, and wait states (complete clock cycles) are inserted until one of the bus cycle termination is met.

- STATE 5 S5 is a low period of the clock, during which the processor does not alter any signal.
- STATE 6 S6 is a high period of the clock, during which data for a read operation is set up relative to the falling edge (entering S7). Parameter #27 defines the minimum period by which the data must precede the falling edge. For a write operation, the processor changes no signal during S6.
- STATE 7 On the falling edge of the clock entering S7, the processor latches data and negates $\overline{AS}$ and $\overline{UDS}$, $\overline{LDS}$, and/or $\overline{DS}$ during a read cycle. The hold time for these strobes from this falling edge is specified by parameter #12. The hold time for data relative to the negation of $\overline{AS}$ and $\overline{UDS}$, $\overline{LDS}$, and/or $\overline{DS}$ is specified by parameter #29. For a write cycle, only $\overline{AS}$ and $\overline{UDS}$, $\overline{LDS}$, and/or $\overline{DS}$ are negated; timing parameter #12 also applies.

On the rising edge of the clock, at the end of S7 (which may be the start of S0 for the next bus cycle), the processor places the address bus in the high-impedance state. During a write cycle, the processor also places the data bus in the high-impedance state and drives $\overline{R/\overline{W}}$ high. External logic circuitry should respond to the negation of the $\overline{AS}$ and $\overline{UDS}$, $\overline{LDS}$, and/or $\overline{DS}$ by negating $\overline{DTACK}$ and/or $\overline{BERR}$. Parameter #28 is the hold time for $\overline{DTACK}$, and parameter #30 is the hold time for $\overline{BERR}$.

Figure 5-35 shows a synchronous read cycle and the important timing parameters that apply. The timing for a synchronous read cycle, including relevant timing parameters, is shown in Figure 5-36.

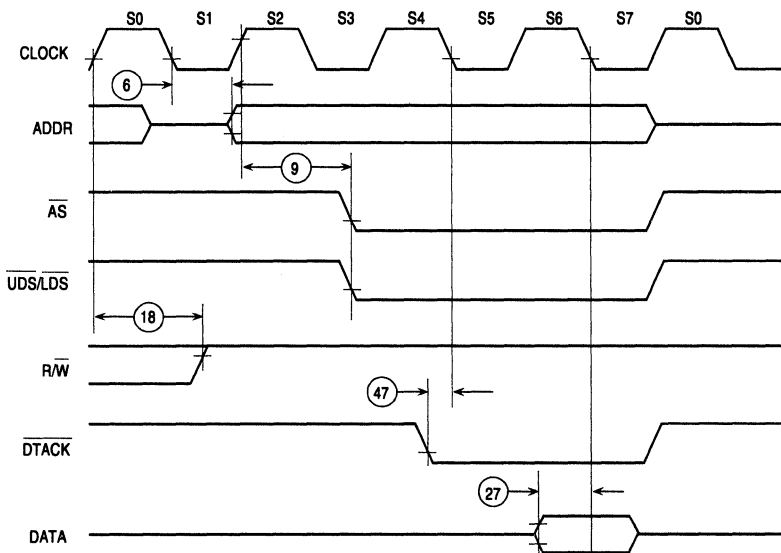


Figure 5-35. Synchronous Read Cycle

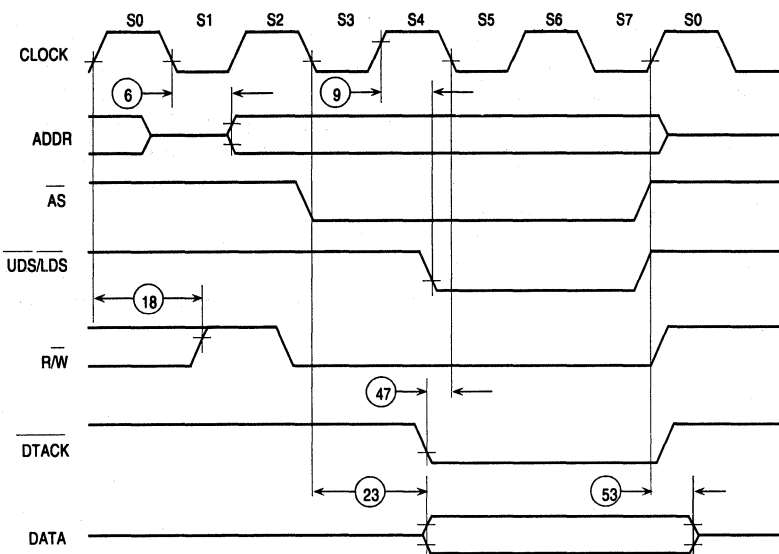


Figure 5-36. Synchronous Write Cycle

A key consideration when designing in a synchronous environment is the timing for the assertion of DTACK and BERR by an external device. To properly use external inputs, the processor must synchronize these signals to the internal clock. The processor must sample the external signal, which has no defined phase relationship to the CPU clock, which may be changing at sampling time, and must determine whether to consider the signal high or low during the succeeding clock period. Successful synchronization requires that the internal machine receives a valid logic level (not a metastable signal), whether the input is high, low, or in transition. Metastable signals propagating through synchronous machines can produce unpredictable operation.

Figure 5-37 is a conceptual representation of the input synchronizers used by the M68000 Family processors. The input latches allow the input to propagate through to the output when E is high. When low, E latches the input. The three latches require one cycle of CLK to synchronize an external signal. The high-gain characteristics of the devices comprising the latches quickly resolve a marginal signal into a valid state.

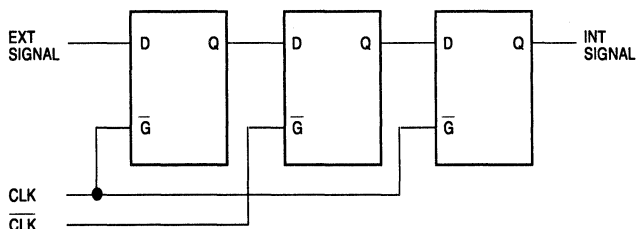


Figure 5-37. Input Synchronizers

Parameter #47 of **Section 10 Electrical Characteristics** is the asynchronous input setup time. Signals that meet parameter #47 are guaranteed to be recognized at the next falling edge of the system clock. However, signals that do not meet parameter #47 are not guaranteed to be recognized. In addition, if $\overline{DTACK}$ is recognized on a falling edge, valid data is latched into the processor (during a read cycle) on the next falling edge, provided the data meets the setup time required (parameter #27). When parameter #27 has been met, parameter #31 may be ignored. If $\overline{DTACK}$ is asserted with the required setup time before the falling edge of S4, no wait states are incurred, and the bus cycle runs at its maximum speed of four clock periods.

The late $\overline{BERR}$ in an MC68010 that is operating in a synchronous mode must meet setup time parameter #27A. That is, when $\overline{BERR}$ is asserted after $\overline{DTACK}$, $\overline{BERR}$ must be asserted before the falling edge of the clock, one clock cycle after $\overline{DTACK}$ is recognized. Violating this requirement may cause the MC68010 to operate erratically.

